

FRED TALKS

2nd September 2026

CONTENTS

Control the ideas, not the code

ANTIREZ.COM · 1245 WORDS

Zen of AI Coding

YOAV AVIRAM · 1912 WORDS

8 Predictions for the Era of Continual Learning

DWARKESH PATEL · 1412 WORDS

Ways to think about token pricing

BENEDICT EVANS · 2607 WORDS

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · [SOURCE](#)

Look at the past history of this blog. There are many blog posts about

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even **not** accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn't y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · [SOURCE](#)

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to [Zen of Python](#) by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead

2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world
11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus's law states that “given enough eyeballs, all bugs are shallow.” Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How “dense” are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is trickier then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering

We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy

I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of [WhiteBoar](#), where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.

8 Predictions for the Era of Continual Learning

DWARKESH PATEL · 07 AUG 2026 · [SOURCE](#)



Locking in AI safety regulation now is a mistake.

I have explained [elsewhere](#) why I think continual learning is needed. I don't think you can have AIs that perform whole jobs as competently as humans if they're forced to just write Markdown files from session to session.

To give an illustrative example, imagine if the way students had to learn to play the saxophone is that one student tries to play it from a cold start, then after her first session, writes down a bunch of notes, then the next student waiting outside the music hall who's also never played the saxophone reads all their notes before trying to play, and so on. Even if you had an infinite sequence of saxophone-virgin students waiting outside the studio that could write notes to the next guy, there's no sequence of text they could write together that would allow the Nth student outside to play proficiently on their first try. At some point, you have to accumulate the experience into the brain. I think the same will be true about a lot of skills and knowledge that we'll want AIs to learn in all the different workplaces they find themselves deployed in.

Okay, so what changes about AI once we have continual learning?

- A lot of the proposals that have been put forward for regulating AI assume that you train a model, and then you deploy it. And therefore if we run a bunch of checks before the model is deployed, then we can make sure that it's not going to aid in cyber attacks or recursive self improvement. But what if the base model is getting updated every single day based on the millions of sessions of work it does? This is one of many reasons why I think it's unwise to lock in some kind of regulatory safety regime right now. We simply don't know what kind of technology we're going to be looking at even in a year, let alone in five years or ten years, and we'd be entrenching an archaic and potentially counterproductive approach to dealing with the threats from AI. To the extent that the government really wants to do some kind of safety evaluations on model providers, it would make more sense to do monthly or quarterly risk inspections rather than singling out some special moment that occurs after training is done and before deployment begins, because that will not be a meaningfully distinct category in the future.
- How the labs do technical alignment would need to totally change. Almost all current techniques are focused on the problem of how we make it so that a frozen set of weights behaves well during deployment. I'm not aware of much research on the question of how to guarantee that, even with constant weight updates, the AI system never falls prey to jailbreaks or changes into a deceptive or evil persona. And if AIs are agglomerating learnings between users as well, how do you prevent users from injecting backdoors or some kind of malicious inclination into the base model? In some sense it is actually closer to the human alignment problem - your kids go out and learn new things, sometimes get one-shotted by crazy ideologies or drugs or something - but you hope you've given them enough common sense and basic values to improve as people in a self-directed way, without ending up with some super weird and misanthropic beliefs.
- The diversity of AI minds will increase. Right now, there are <5 prominent AI minds, and they are all quite similar to each other on account of being trained on roughly the same data. But if AIs are learning from experience, and that experience is different between different AIs, we could see actually different AIs come out the other end.
- When deployment becomes part of training, the returns to being ahead will accelerate. If you have the best model, and more people use your AI for more complicated and useful work, and give it lots of feedback that it can integrate beyond the session window, then your model will become even smarter.

- If the model learns mainly from deployment, labs will feel the pressure to deploy their smartest model earlier. Anthropic has been using Mythos internally since February. But it only shipped this model to the public in June. In a continual learning regime, a four-month internal/external gap means ceding four months of deployment learning. Your competitor who ships earlier might be worse than yours on release day, but it gets to use a lot more real world experience to get better.
- Continual learning will create a clear moat for leading AI labs that they currently lack. Many people have been asking, how will the AI labs actually make money? When I asked Dario this question on my podcast, he made the analogy to cloud providers, who also offer many undifferentiated services, but earn high profit margins on them nonetheless. But the reason cloud margins are high is that it's really time-consuming and expensive to switch from one cloud to another. But currently there is no switching cost for AI models. There's nothing that's preventing me from starting a software repository with Codex and then finishing it with Claude Code. But once we get continual learning and the model you're working with is actually getting better as it interacts with you from session to session, then there are actually pretty significant switching costs. If you want to change what AI you are using, you basically have to fire an employee that has months of context on your organization and replace them with a fresh one that you have to retrain from scratch. Once you're locked in like this, model providers can demand pretty hefty margins.
- Enterprises will be wise to this. They will try to avoid this kind of lock-in. But what if the choice is that you either get locked into a model provider, or you lose out on this super valuable feature where the model improves for you session after session? If real usage ends up becoming the main way models improve, then labs may subsidize users and enterprises which allow the model to train on their sessions, especially on hard economically important work. Just the same way that Google gives away search. And conversely, the labs may say that any enterprise that refuses to let them train on its sessions can't have access to the very best models. With both carrots and sticks, the labs will try to get their users to allow AIs to learn from experience. I'm glossing over the fact that there's a difference between updating one user's set of weights, and pooling all these different weight forks back into the main model, and the latter may be technically more challenging, but in due time that too will be solved.
- AI training already has large economies of scale (the theoretical reason to expect this is that you can amortize your expensive training across more users, and the practical evidence is that so far, lab revenues have increased faster than their compute). But continual learning may also lead to economies of scale in inference for end users, namely from batching, if per-company information requires full weight updates rather than living in low rank adapters. Back-of-the-envelope math suggests that the optimal

inference batch size for a sparse model like DeepSeek V3 is $> 2,400$ (that is to say that unless your model is concurrently generating that many sequences at once, you're underutilizing your compute). If you want to understand why, go watch my [full episode with Reiner Pope](#) on inference economics. But anyway, the point here is that a given set of weights is only served efficiently when thousands of sequences are decoding against it at once. A large company whose employees and agents generate that much concurrent traffic can efficiently serve their continually-updated weight fork; an individual user serving themselves at batch size 1 might suffer a 100x+ compute efficiency penalty. So the economics of serving personalized weights strongly favor big organizations.

Plenty more will have changed by the time continual learning works, and the most important changes are probably the ones hardest to anticipate. But the ones above seem clear even now.

Upgrade to paid

[View in app](#)

Ways to think about token pricing

BENEDICT EVANS · 09 AUG 2026 · [SOURCE](#)

AI is in a supply crunch today, but what happens when we come out of it? How and where will supply, demand, price, capacity and capex get back into equilibrium? Today, model labs can name their price, but why won't they end up as low-margin commodity infrastructure?

There are only two things you can say with certainty about token prices: we're in a supply crunch, and this is unstable. All of the variables are in play, and the market will get shaken out over the next few years to arrive at a new equilibrium. Right now we have a lot of frantic analysis of 'time to power', but question at the end of that is whether the foundation models

have sustainable pricing power, strategic leverage and value capture, or whether they become low-margin commodity infrastructure providers. At the moment, I think every dynamic we can see points to the latter.

Clearly, the situation today is transitory. On the supply side, a trillion dollars or more of data centre capex is coming down the pipe (and plenty more semiconductor capex behind that), inference efficiency continues to improve very quickly, and new models are far more (or far less!) efficient in their token use. On the demand side, although the market has been capacity-constrained since 2022, the crunch in the first half of this year has been driven by sudden product-market fit in really just one use case, software development, and that's actually a pretty small field (imagine if we had product-market fit for a consumer use case with hundreds of millions of DAUs - today's infrastructure couldn't support it at any price). We don't know what the next use-cases to scale will be, nor when that would be, nor what their token needs would be.

Going up a level, it's been pretty widely reported that inference today has 40-50% gross margins: this includes depreciation of the associated server costs (or the cost of renting them), but we don't really know the asset life (five years? Seven years?) and obviously this doesn't include the cost of training the next model a couple of times a year, which is currently far larger than revenue. In principle, inference is a marginal cost and training is a fixed cost, so with high enough revenue you can reach profitability, but we don't know how training costs will change. On the other side of the table, it's unclear how much of the surge in use in the last few months has an ROI (or at least has an ROI that can be quantified to a CFO), let alone any future use cases, and hence what prices people might be prepared to pay for them.

So, all the variables will move all over the place over the next 12 months, and move again over the next three to five years. How could we suggest where this will settle? How and where will supply, demand, price, capacity and capex get back into equilibrium?

In theory, you can model this bottom-up. You can make some assumptions around each of the variables I suggested above, and then try to model out how many chips there are now, how many chips with what performance TSMC and the rest of the semis industry might be able to deliver when, and how fast all of that can be brought online in data centers and how fast those can be powered. Then you can wonder about price discipline, and make some guesses about use cases. This will get you a number, but it will be rather like trying to build a five-year forecast for the broadband market in 1998: the spreadsheet will be very pretty, and you might even get close to the right number for this year, but there are too many unknown variables to make a useful forecast of a longer-term market structure.

getting mostly the same results, and we don't yet know of a network effect or any other winner-takes-all effect that would let one company pull ahead, stay ahead, and do things that others could not, in some sustainable way. Does that change?

Fourth, how much of the value from those high-end use cases is captured by the frontier model itself? How much needs to be wrapped in tooling, process, proprietary data, go-to-market, networks, support, and everything else associated with a traditional software company, even if you do need the big expensive frontier model underneath? Can that model do the whole thing, or is the model, no matter how good, still a piece of infrastructure that you use to make the actual product? At the extreme, can the model itself invent and make all of those things, and would that let them charge by seat, by outcome or just take the profit? Or do even (or especially) the most sophisticated and high-value use-cases need to sit inside hundreds of new companies that can pick and choose which models to use?

None of these are binaries: they're all a question of degree, and they'll probably vary quite a lot by use case. But at one extreme, there are two or three giant minds that run half of everything and have massive pricing power, and at the other extreme LLMs look like databases - there'll be millions of them, some very big and some very small, and the value is in what you build on top - after all, every SaaS company is a 'database wrapper'. There's a future in which Anthropic (or a company we haven't heard of yet) wins the whole thing and can set its own terms, and a future in which dozens of routers run real-time auctions to allocate your tasks across hundreds of low-margin model-farms and a benchmark company takes a fee on every single one.

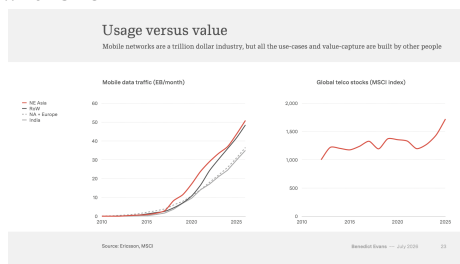
I don't think anyone can actually *know* the answer yet. I've said "we don't know" a lot, and that's very deliberate. Part of the concept of the 'S Curve' is that there's a stage early in the emergence of a new technology where it's clear that this is going to be huge but nothing else is clear at all - the mid 1990s for the internet, say, and 2008 or 2009 for mobile. There are places where you can take a view - for example, I've argued at length that I think chatbots are a poor interface that will struggle to capture value up the stack - but we should presume there are big questions that we can't see yet, let alone answer, and anyone picking one of the ten possible outcomes we can see and saying "it will be that one!" is just guessing.

Meanwhile, there is structural uncertainty at the early stages of every big new technology, but the uncertainty now is different, because we don't have a good theoretical understanding of why these models work so well and so we don't know how much better they can get. In 1995, we didn't know how the internet would evolve but we knew that there were less than 100m PCs on earth (and they were expensive) and that telcos couldn't give everyone FTTH next year; in 2010 we didn't know what the next iPhone would be but we knew it wouldn't have retinal projection. We knew the physical limits in ways we don't really know with LLMs. Next month a new approach could cut inference compute needs by 90%, or double demand, or both.

All of this has people hunting for patterns to recognise (indeed, it's been observed that *all* conversations about AI end in a hunt for metaphors). It has become common to draw comparisons with fiber, which had a massive overbuild in the Dotcom bubble that looks a bit like the infrastructure build-out today. The narrow problem with that is that fiber construction was far ahead of demand, where AI compute construction is far behind demand, though, as above, we don't know what that supply/demand balance will look like in the future. But the more relevant objection, I think, is that fiber construction was mostly fixed cost (digging holes) rather than marginal cost (more equipment), whereas growth in compute needs means you need to buy more compute.

That makes mobile data a more fruitful comparison here. Mobile networks have marginal cost for capacity, and like AI they had an enormous surge in usage 15 years ago, that overwhelmed capacity and had carriers scrambling to add capacity and rebalance their pricing. Meanwhile, selling bits looks superficially similar to selling tokens: it's an opaque measure of marginal cost that doesn't map in any transparent or intuitive way to use cases or value, and needs to be replaced with bundles of some kind. But most importantly, in the last 20 years cellular data traffic has risen by several orders of magnitude, and this has become an enormous industry, with annual revenue of a trillion dollars and capex of \$200 billion, but the stocks have gone nowhere, and all the value was captured by other people further up the stack. This, of course, is one of the core questions for AI: is this going to be low-margin commodity infrastructure with all the value captured by other people further up the stack?

[View fullsize](#)



Semiconductor manufacturing also has echoes of AI, since unlike cellular it has something of the same escalating cost and complexity that we see in foundation models. Rock's Law pointed out that the cost of a cutting edge fab doubles every four years: over time the frontier of semiconductors become so hard and so expensive that the number of players dropped from dozens to a handful and now really only one, TSMC. That's another core question for AI: will this get so hard and so expensive that only a couple of people can do it, even without network effects? Equally, many semiconductor uses are further back along the same kind of price/

performance curve I just discussed for AI. Even here, though, while TSMC has a *de facto* monopoly on the frontier, and nice margins, it doesn't actually capture a large share of value from the broader tech economy: net income last year was \$53bn, less than half of Apple alone.

There are plenty of other comparisons one could make here: in the last six months Sam Altman has compared OpenAI both to Windows, a high-margin capital-light monopoly based on network effects, and electricity utilities, which are natural monopolies but also low-margin regulated utilities selling a pure commodity. One could also point to cloud (three leading players with good margins and clearly distinguished propositions, but again limited value-capture). But none of these have predictive value: analogies don't have predictive value. You can't prove whether something will have the same outcome as mobile by arguing how much it's like mobile: this was a mistake that many very clever people in tech made 15 years ago, claiming that Android would beat iOS because Android was 'open' and 'open' Wintel had beaten the 'closed' Mac in the 1990s (it's also the mistake that doomers make when they claim that AI is 'like' nuclear weapons). Everything is different: bits, tokens and transistors are different, each of these examples are different from each other, and AI will be different too.

However, these examples do tell us, empirically, that something can be very important, very expensive, change the world, and be full of very sophisticated science and engineering, and yet have a wide range of possible outcomes. There isn't one inevitable path here: you can have price equilibrium at high margins and at low margins, and with and without market concentration, and you can't hand-wave that away by talking about AGI and saying "you don't understand exponentials!"

However, if one thread in everything I've written above is how much we don't yet know, the other thread is that every path to foundation models having market dominance, strategic leverage, value capture, winner-takes-all effects, or anything else other than becoming commodity infrastructure, requires something to change.

Maybe frontier models will become less competitive - yet in the last six months, Mark Zuckerberg and Elon Musk jumped from zero back onto the leaderboards. Maybe network effects will emerge. Maybe chatbots can grow into products and don't need to be wrapped in software. Maybe one lab will start out-executing all the others and pull ahead on sheer product dynamism - Microsoft, Google, Facebook and Apple had to execute their way into a wining position before they had winner-takes-all efforts. Maybe *something* else will happen.

In particular, we have not one but two potential *dei ex machina* - Trump and China. China is reportedly considering regulating open source and some people close to Trump have floated this as well (though since Meta abandoned Llama the US has no leading open model), and export

controls could expand and become systematic. Many people see the pleas for regulation from Anthropic and (sometimes) OpenAI as a front for regulatory capture, but either way, we can't presume this will remain an entirely free market.

Even so, that brings me back to the same point: the current market dynamics point to a future in which, as today's supply crunch eases, frontier models move towards becoming commodity infrastructure, with all of the value built on top, and for a different outcome, something needs to happen that we don't see yet.